



TUTORIAL DO ALUNO

Criando e entregando uma xApp KPM com Nephio

Da ideia ao KPI no Grafana, com deploy no NMI e experimento ARM no CIC.

OBJETIVO Ensinar o aluno a programar, empacotar, implantar, estimular e avaliar uma xApp de monitoramento em uma Open RAN de laboratório.

NMI + CIC | PoC acadêmica | versão 1.0 | 31 de agosto de 2026
Repositório: github.com/lucasrodri/oran-stack

0 Como usar este tutorial

Este roteiro é um laboratório guiado, não um manual de produção. Ele parte de uma pilha Open RAN já implantada e saudável e leva o aluno até uma xApp KPM observável e entregue por GitOps.

TEMPO ESPERADO 60 a 90 minutos quando a infraestrutura base, a CI e as credenciais já estão prontas.

Resultado final

- uma xApp Python com um algoritmo pequeno e mensurável;
- uma imagem imutável compatível com amd64 e arm64;
- um Team Blueprint e uma variante NMI publicados pelo Nephio/Porch;
- um Deployment entregue pelo Flux com identidade restrita;
- um experimento reproduzível de tráfego do UE e evidência no Grafana;
- uma atualização e um rollback para frente, sem apagar histórico.

Mapa do roteiro

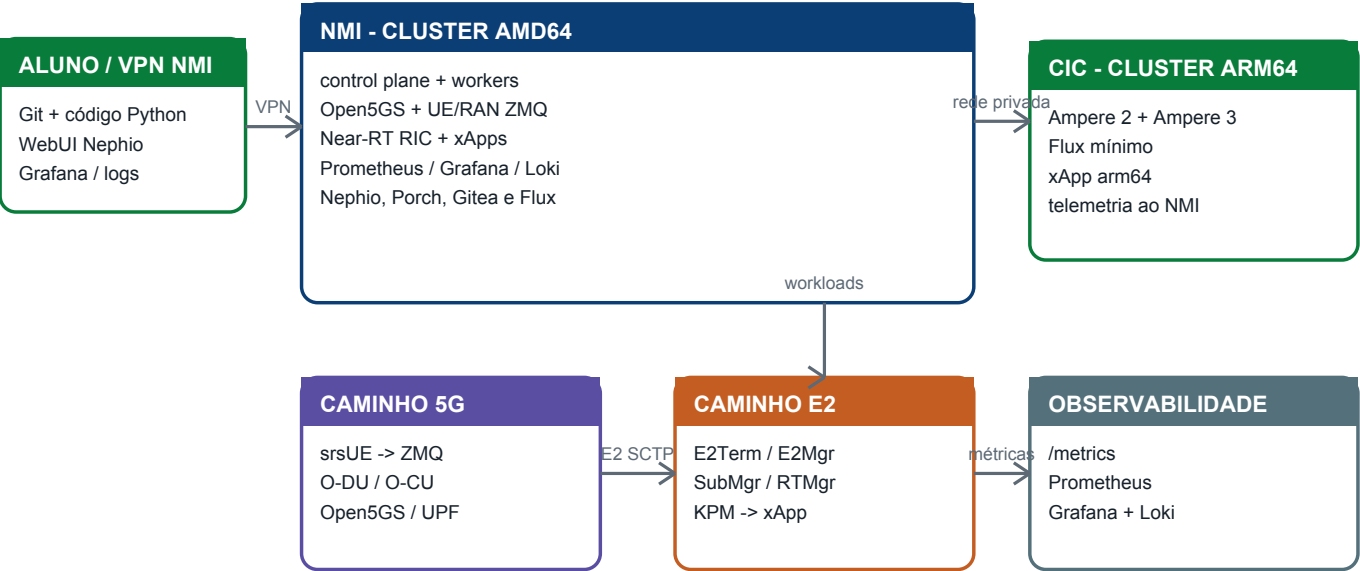
Etapa	O que acontece	Responsável principal
1-3	entender arquitetura, validar base e gerar scaffold	aluno
4-5	programar, testar e publicar imagem	aluno + CI
6-7	publicar pacote e entregar pelo Flux	aluno assistido + infra
8-9	validar KPM e estimular UE	aluno
10	variante ARM e bridges RMR	infra assistindo o aluno
11-13	update, rollback, diagnóstico e relatório	aluno

REGRA DO LABORATÓRIO Se RAN, E2 ou UE não estão saudáveis, pare. Não trate uma falha da pilha base como defeito da nova xApp.

1 Arquitetura do laboratório

A PoC usa dois clusters porque há duas arquiteturas de CPU e dois locais físicos. O NMI concentra a pilha de telecom, o management plane Nephio e a observabilidade. O CIC executa workloads ARM e recebe configuração pelo Flux.

Topologia didática multi-site

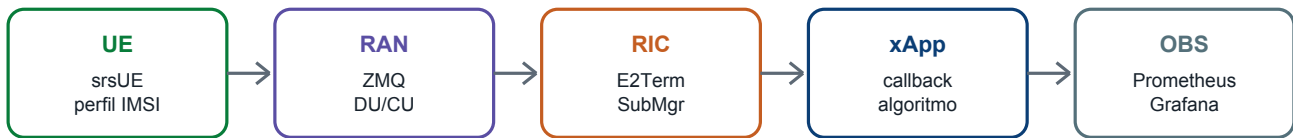


Plano de gestão: Git -> Porch -> Gitea -> Flux -> Kubernetes
Plano de dados: UE -> RAN -> UPF -> Internet
Plano de controle/telemetria: O-DU -> E2 -> Near-RT RIC -> xApp -> Grafana

Figura 1 - Topologia lógica da PoC acadêmica NMI/CIC.

Site	Arquitetura	Papel no laboratório
NMI	amd64	Open5GS, RAN/UE ZMQ, Near-RT RIC, observabilidade, Nephio/Porch/Gitea e xApps locais
CIC	arm64	cluster Ampere, Flux mínimo, xApps remotas e telemetria enviada ao NMI

2 Modelo mental: quem faz o quê



A autenticação do assinante acontece no Open5GS; a xApp não cadastra SIM.

O KPI DRB.UETHpDI é medido no O-DU e chega à xApp como indicação E2SM-KPM.

O teste gera tráfego no tun_srsue e observa a mudança no dashboard.

Figura 2 - Caminhos independentes de autenticação, rádio, telemetria e observabilidade.

UE, SIM e Open5GS

O UE representa o aparelho. O IMSI identifica o assinante/SIM e é provisionado no Open5GS com suas credenciais de autenticação. O IMEI identifica o equipamento. A xApp não cadastra SIM, não autentica UE e não substitui o Core.

KPI, KPM e xApp

KPI é um indicador de desempenho. KPM é o serviço E2 que transporta medições da RAN. A xApp assina uma métrica, recebe indicações pelo RMR, executa o algoritmo do aluno e expõe séries Prometheus.

Nephio não é Helm nem Rancher

Nephio gerencia pacotes declarativos e suas variantes por site. Porch publica revisões; Gitea guarda a configuração resultante; Flux aplica essa configuração no cluster. Kubernetes continua executando os pods, e o RIC continua cuidando das funções E2/RMR.

NESTA POC Nephio roda em namespaces próprios no cluster NMI, agendado no worker nephio-k8s-w01. Ele não é um segundo cluster Kubernetes.

3 Pré-voo: validar a pilha antes de programar

Conecte a VPN NMI, atualize sua branch a partir de main e use um nome DNS seguro, com letras minúsculas, números e hífens.

Acesso inicial

```
ssh aluno@192.168.72.10
sudo kubectl get nodes -o wide
sudo kubectl -n near-rt-ric get pods
sudo kubectl -n ran get pods
curl -fsS http://127.0.0.1:30380/v1/nodeb/states
```

Critérios mínimos

- E2Term, E2Mgr, SubMgr, RTMgr e AppMgr estão Ready;
- o nó gnbd_001_001_00019b_e2 aparece como CONNECTED;
- O-DU, O-CU e srsUE estão Ready;
- a interface tun_srsue possui endereço;
- Prometheus e Grafana estão acessíveis.

Interfaces pela VPN

Serviço	URL
Nephio WebUI	http://192.168.71.30:30707/config-as-data
Grafana - visão geral	http://192.168.72.10:30300/d/oran-overview/o-ran-stack-overview
Grafana - logs	http://192.168.72.10:30300/d/oran-logs/o-ran-kubernetes-logs
Alertmanager	http://192.168.72.10:30301

PARE AQUI SE o E2 node estiver DISCONNECTED, o UE não tiver tun_srsue ou os contadores da xApp de referência estiverem congelados.

4 Gerar a nova xApp

O gerador cria o ponto de entrada Python, o pacote reutilizável Nephio, a variante NMI e a identidade Flux. Ele se recusa a sobrescrever uma xApp existente.

```
git switch main
git pull --ff-only
git switch -c aluno/kpm-latency-lab

./scripts/new-kpm-xapp.sh kpm-latency-lab DRB.UETHPD1 2000
```

Artefatos criados

Caminho	Finalidade
xapps/python/kpm_latency_lab_xapp.py	código executado no container
packages/nephio/kpm-latency-lab/	Team Blueprint reutilizável
infra/nephio/blueprints/*-nmi-variant.yaml	especialização para o NMI
infra/nephio/nmi-onboarding/*-rbac.yaml	permissões restritas da entrega
infra/nephio/nmi-onboarding/*-flux-sync.yaml	reconciliação GitOps

Escolher métrica e período

O primeiro laboratório usa DRB.UETHPD1, throughput downlink observado no O-DU. Use um período diferente de outra xApp concorrente que peça a mesma métrica. Solicitações idênticas podem ser fundidas pelo SubMgr.

EXEMPLO r4-simple-mon NMI usa 1000 ms, simple-mon CIC usa 2000 ms e kpm-load-watch usa 3000 ms.

5 Programar e testar o comportamento

O ponto de extensão é `indication_callback`. xAppBase já cuida do registro no AppMgr, assinatura no SubMgr, loop RMR, decodificação KPM e métricas Prometheus.

Exemplo de algoritmo seguro

```
def indication_callback(self, e2_agent_id, subscription_id,
                        indication_header, indication_message):
    measurements = self.e2sm_kpm.extract_meas_data(indication_message)
    value = measurements.get("measData", {}).get("DRB.UETpDl", 0)

    self.window.append(float(value or 0))
    average = sum(self.window) / len(self.window)
    state = "busy" if average >= 20000 else "active" if average >= 1000 else
           "idle"
    self.export_state(average, state)
```

BOA PRÁTICA Mantenha o callback rápido e não bloqueante. Exporte valores em `/metrics` e limite os logs; não escreva uma linha para cada indicação.

Métricas padrão já disponíveis

- `oran_xapp_active_subscriptions`;
- `oran_xapp_rmr_indications_total`;
- `oran_xapp_kpm_indications_total`;
- `oran_xapp_kpm_measurement`;
- `oran_kpm_drb_ue_throughput_dl_kbps` para `DRB.UETpDl`.

Validação local

```
python3 -m py_compile xapps/python/kpm_latency_lab_xapp.py
python3 -m unittest discover -s tests -v
bash -n scripts/new-kpm-xapp.sh
git diff --check

rg 'REPLACE_WITH_MULTIARCH_IMAGE_DIGEST' packages/nephio/kpm-latency-lab
infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml
```

6 Construir a imagem multiarch

O Dockerfile comum inclui toda a pasta xapps/python. O aluno não precisa criar um Dockerfile por xApp. Depois do push, a CI deve publicar um índice com as plataformas linux/amd64 e linux/arm64.

```
git add xapps/python/kpm_latency_lab_xapp.py    packages/nephio/kpm-latency-lab
infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml    infra/nephio/nmi-
onboarding/kpm-latency-lab-*.yaml

git commit -m "feat(xapp): add kpm latency lab monitor"
git push -u origin aluno/kpm-latency-lab
```

Fixar pelo digest do índice

Nunca use :latest e não copie o digest de apenas uma arquitetura. A referência deve apontar para o índice multiarch imutável.

```
IMAGE='ghcr.io/lucasrodri/oran-stack/oran-xapps@sha256:<digest>'

# Substitua o marcador nos dois arquivos gerados:
# packages/nephio/kpm-latency-lab/workload-cluster.yaml
# infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml

rg 'REPLACE_WITH|:latest' packages/nephio/kpm-latency-lab    infra/nephio/blueprints/kpm-
latency-lab-nmi-variant.yaml
```

ACEITE O último comando não retorna nada, e a CI confirma amd64 + arm64 para o mesmo digest do índice.

7 Publicar com Nephio e entregar com Flux



O aluno produz código, teste, imagem e blueprint.

A infraestrutura aprova publicação, RBAC, portas e variantes remotas.

Nephio renderiza configuração; Flux realiza a entrega no cluster escolhido.

Figura 3 - Pipeline declarativo usado na PoC.

No worker Nephio

```
ssh -J aluno@192.168.72.10 aluno@192.168.71.30
cd /caminho/para/oran-stack
export KUBECONFIG=/etc/nephio/nmi-admin.conf

sudo -E ./infra/nephio/blueprints/publish-xapp-blueprint.sh kpm-latency-lab
packages/nephio/kpm-latency-lab v1

sudo -E kubectl apply -f infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml

sudo -E ./infra/nephio/blueprints/approve-packagevariant.sh kpm-latency-lab-nmi
```

Iniciar a reconciliação

```
sudo -E kubectl apply -f infra/nephio/nmi-onboarding/kpm-latency-lab-rbac.yaml
sudo -E kubectl apply -f infra/nephio/nmi-onboarding/kpm-latency-lab-flux-sync.yaml

sudo -E kubectl -n flux-system wait kustomization/kpm-latency-lab --for=condition=Ready
--timeout=300s
```

O QUE O NEPHIO FEZ Blueprint + intenção de site produziram um pacote NMI publicado. O Porch não executou a xApp; o Flux aplicou o pacote no Kubernetes.

8 Aceite funcional da xApp

Uma xApp não está aceita apenas porque o pod está Running. É preciso provar registro, assinatura, recebimento RMR, decodificação KPM e observabilidade.

```
sudo kubectl -n ricxapp rollout status deployment/kpm-latency-lab --timeout=180s

sudo kubectl -n ricxapp get pod,service -l app=kpm-latency-lab -o wide

curl -fsS http://127.0.0.1:30080/ric/v1/xapps | grep -F 'kpm-latency-lab'

sudo kubectl -n ricxapp exec deployment/kpm-latency-lab -c xapp -- curl -fsS
http://127.0.0.1:8091/metrics | grep -E
'^oran_xapp_(active_subscriptions|rmr_indications_total|kpm_indications_total)'
```

Checklist de aceite

- [] pod Ready, sem aumento contínuo de restarts;
- [] AppMgr lista o endpoint RMR correto;
- [] active_subscriptions = 1;
- [] contadores RMR e KPM aumentam entre duas leituras;
- [] RMR_FLAGS=1 e um único listener TCP em 4561;
- [] Prometheus descobre o Service com monitoring=true;
- [] Grafana separa a série pelo label service.

IMPORTANTE Running prova apenas que o processo iniciou. O aceite científico exige um estímulo conhecido e uma resposta mensurável.

9 Gerar tráfego e enxergar o KPI

O estímulo é uma transferência limitada de 50 MB pela interface tun_srsue. O tráfego cruza UE -> RAN -> UPF -> Internet, enquanto a xApp recebe DRB.UEThpDI pelo E2.

```
cd /home/aluno/oran-stack-main
sudo env KUBECONFIG=/etc/kubernetes/admin.conf XAPP_DEPLOYMENT=kpm-latency-lab
./scripts/demo-kpm.sh
```

O que observar no Grafana

- assinatura ativa igual a 1;
- taxa de indicações KPM acima de zero;
- pico de DRB.UEThpDI durante a transferência;
- retorno do KPI para próximo de zero após o tráfego;
- se houver classificador: transição idle -> active -> busy -> active -> idle.

Demonstração de referência

```
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-load-watch.sh
```

```
# Sucesso esperado:
DEMO_LOAD_WATCH_OK
```

Na validação de referência, kpm-load-watch calculou média móvel de cinco amostras, atingiu pico de aproximadamente 28.841 kbps e completou o ciclo de estados.

SE O KPI FICAR ZERO Confirme que o HTTP retornou 200, que o curl usou tun_srsue, que o UE tem PDU session e que a métrica é implementada pelo E2 node.

10 Perfis de UE no laboratório

O laboratório possui três perfis de assinante, mas usa um por vez no único enlace ZMQ. Isso equivale a trocar o SIM/aparelho conectado ao rádio virtual; não equivale a três UEs simultâneos.

Perfil	IMSI	Uso
ue1	001010000000001	referência
ue2	001010000000002	aluno / experimento A
ue3	001010000000003	aluno / experimento B

Trocar o perfil e executar a demonstração

```
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-ue-lab.sh ue2

# repetir com outro assinante
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-ue-lab.sh ue3

# voltar à referência
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/select-ue-lab-profile.sh ue1
```

LIMITAÇÃO ATUAL DRB.UETpDI Report Style 1 é agregado no O-DU. Ele prova carga de rádio, mas não atribui a medição a um IMSI específico.

Por que não escalar srsUE para três réplicas?

O DU atual oferece um único par tx_port/rx_port ZMQ. Réplicas concorrentes disputariam o mesmo enlace e criariam uma topologia inválida. Multi-UE real requer rádio, emulador ou broker RF apropriado.

11 Variante ARM no CIC

Depois que a xApp funciona no NMI, a mesma imagem multiarch pode ser especializada para o CIC. O container roda nativamente como aarch64, enquanto RIC, RAN, UE e Core permanecem no NMI.

O que muda

- PackageVariant injeta site=cic, architecture=arm64 e o nó Ampere;
- Flux no CIC puxa o repositório de deployment publicado no NMI;
- NodePorts e bridges RMR conectam a xApp remota ao RTMgr/E2Term no NMI;
- Prometheus no NMI recebe a telemetria da xApp CIC por um relay sem logs.

Teste de aceite multi-site

```
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-kpm-multisite.sh
```

```
# Sucesso esperado:  
DEMO_KPM_MULTISITE_OK
```

Verificação	Resultado esperado
arquitetura do processo	aarch64 no cic-k8s-w01
Flux	Kustomization Ready=True
RMR	um listener receive-only e retorno ao RTMgr
KPM	contadores avançam no NMI e CIC
Grafana	séries locais e cic-simple-mon no mesmo painel

AÇÃO DA INFRA O aluno não escolhe NodePorts nem cria regras no pfSense. Cada xApp remota precisa de portas e bridges exclusivas para evitar colisões.

12 Atualização e rollback

Pacotes Published são imutáveis. Atualização e rollback criam novas revisões para frente; o histórico Git e Porch permanece auditável.

Atualização

- 1. altere o algoritmo e acrescente um teste;
- 2. gere uma nova imagem e fixe o novo digest;
- 3. publique a revisão v2 do Team Blueprint;
- 4. atualize o PackageVariant para workspaceName: v2;
- 5. aprove o downstream, espere Flux Ready e repita o experimento.

Rollback para frente

Copie o conteúdo conhecido de v1 para uma nova revisão, por exemplo v3-rollback. Não use git reset, não mova tags publicadas e não edite uma PackageRevision Published.

DEMONSTRAÇÃO O repositório inclui demo-xapp-update-rollback.sh para a xApp de referência. Ele cria uma atualização inofensiva e um rollback para frente.

Evidências a guardar

- revisões Porch de origem e downstream;
- commit Git consumido pelo Flux;
- digest da imagem;
- status do rollout;
- assinatura, contadores KPM e gráfico antes/durante/depois.

13 Diagnóstico rápido

Sintoma	Primeira verificação
init container esperando	E2 node ausente ou DISCONNECTED no E2Mgr
registro existe, assinatura falha	nome/endpoint do Service e resposta HTTP do SubMgr
KPM congelou após DU	nova associação E2; recriar somente pods stateless das xApps
contadores avançam, KPI zero	UE ocioso, PDU, tun_srsue ou métrica não implementada
duas xApps compartilham ID	métrica e período idênticos; escolher período distinto
RMR intermitente	RMR_FLAGS=1 e exatamente um listener em 4561
Flux Ready=False	Git, RBAC, digest e manifesto renderizado
funciona NMI, falha CIC	índice sem arm64 ou dependência nativa incompatível
Grafana No Data	Service monitoring=true, target Prometheus e janela de tempo

Recuperação controlada do ZMQ

```
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/recover-ran-zmq.sh
```

Depois de confirmar o gNB CONNECTED, recrie somente os pods stateless das xApps para abrir novas assinaturas. Não reinicie o E2Term se a associação SCTP já estiver saudável.

Logs sem exagero

O laboratório usa retenção curta. Consulte logs por namespace, pod e janela de tempo. Prefira métricas para séries temporais e limite mensagens no callback a uma amostra periódica.

14 Entrega do experimento

A entrega deve permitir que outra pessoa reproduza o resultado e entenda o que a xApp acrescentou ao laboratório.

Checklist do aluno

- [] problema ou hipótese descritos em até um parágrafo;
- [] código da xApp e teste unitário no repositório;
- [] logs limitados e nenhum segredo no código;
- [] imagem amd64/arm64 fixada pelo digest do índice;
- [] Team Blueprint e PackageVariant publicados;
- [] Flux Ready com ServiceAccount restrita;
- [] AppMgr, assinatura, RMR e KPM validados;
- [] estímulo do UE reproduzível;
- [] atualização e rollback demonstrados;
- [] gráfico ou tabela com hipótese, estímulo e resultado.

Estrutura sugerida do relatório

Seção	Conteúdo mínimo
Objetivo	pergunta e comportamento esperado
Método	métrica, período, algoritmo, UE e volume de tráfego
Ambiente	site, arquitetura, digest e revisão do pacote
Resultados	KPI, estados, tempos e gráfico
Discussão	limitações, erros e próximos testes
Reprodução	comandos e critérios de aceite

OBJETIVO ACADÊMICO O valor não está apenas em fazer o pod subir. Está em formular uma hipótese, produzir um estímulo controlado e medir uma resposta da RAN.

A Glossário e referência rápida

Termo	Significado neste laboratório
UE	equipamento do usuário; aqui, srsUE simulado
IMSI	identidade do assinante/SIM provisionada no Open5GS
IMEI	identidade do equipamento
O-DU / O-CU	funções distribuída e central da RAN
Near-RT RIC	plataforma que hospeda e conecta xApps à RAN via E2
xApp	aplicação do Near-RT RIC que monitora ou controla a RAN
KPI	indicador de desempenho
E2SM-KPM	modelo de serviço E2 para medições de desempenho
RMR	mensageria usada entre componentes do RIC e xApps
Nephio	automação de pacotes declarativos e variantes por site
Porch	servidor de pacotes e revisões KRM
Flux	reconciliador GitOps que aplica o pacote no Kubernetes

Arquivos canônicos do repositório

- docs/STUDENT_XAPP_LAB.md - procedimento técnico completo;
- docs/UE_SIMULATION_LAB.md - perfis UE e limitações ZMQ;
- scripts/new-kpm-xapp.sh - gerador da xApp;
- scripts/demo-load-watch.sh - experimento de referência;
- infra/nephio/blueprints/ - publicação e variantes;
- packages/nephio/ - Team Blueprints reutilizáveis.

Fonte da identidade visual

Logo oficial da Universidade de Brasília: cen.unb.br/wp-content/uploads/2025/02/logo_unb_oficial_2025.png

PRÓXIMO LABORATÓRIO Crie uma xApp nova a partir deste roteiro, escolha um algoritmo simples e compare seu resultado com kpm-load-watch.